

Reviewer Pack — Minimal Symplectic Leaves Number Bounds Communication Comple...

Entry e38b2feb290c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 16:30:08 UTC

Minimal Symplectic Leaves Number Bounds Communication Complexity for XOR

Entry ID: e38b2feb290c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 16:30:08 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Boolean Function Complexity: Circuit Complexity of XOR

Statement:

For every Boolean function f with n inputs, the number of symplectic leaves $L(f)$ in the moment map of its graph is upper bounded by a function $g(n)$ such that $\log_2(1 + CC_R(f)) \leq O(g(n))$, where $CC_R(f)$ is the randomized two-party communication complexity of f . Equivalently: if there exists a Boolean function f with n inputs and $L(f) > g(n)$, then $CC_R(f) > 2^n$.

Rationale (proposer's reasoning):

Symplectic geometry provides a geometric interpretation of Boolean functions, where leaves represent distinct input-output patterns. A higher number of symplectic leaves could imply more complex communication between parties required to compute the function, thus potentially providing a lower bound on communication complexity.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32b616acab473216

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean function f with n inputs, if the number of symplectic leaves $L(f)$ exceeds an upper bound $g(n)$, then the randomized two-party communication complexity $CC_R(f)$ must be greater than 2^n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (1): - communication complexity AND randomized two-party AND moment map

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def generate_random_boolean_function(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def symplectic_leaves(f):
    n = int(math.log2(len(f)))
    leaves = set()
    for i in range(2**n):
        leaf = tuple(f[i:i+n])
        if leaf not in leaves:
            leaves.add(leaf)
    return len(leaves)

def randomized_two_party_communication_complexity(f):
    n = int(math.log2(len(f)))
    complexity = 0
    for _ in range(10): # Sample 10 random inputs to estimate the complexity
        input_index = random.randint(0, len(f) - 1)
        input_bits = [int(bit) for bit in bin(input_index)[2:].zfill(n)]
        output_bit = f[input_index]
        complexity += sum(input_bits) + output_bit
    return complexity / (n + 1)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_random_boolean_function(n)
        L_f = symplectic_leaves(f)
        CC_R_f = randomized_two_party_communication_complexity(f)
```

```

    if L_f > n: # Upper bound g(n) is n
        return {
            "metric_name": "CC_R(f)",
            "metric_value": CC_R_f,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": f"Function with {n} inputs and {L_f} leaves"
        }

    return {
        "metric_name": "CC_R(f)",
        "metric_value": sum(CC_R_f for _ in range(30)) / 30, # Average over multiple samples
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Function with {result['n_max']} inputs and {result['counterexample']} leaves\"")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

with 5 inputs and 24 leaves'}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 5.333333333333333, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 4.5, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 5.0, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 6.333333333333333, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 6.333333333333333, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 4.833333333333333, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 5.5, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 4.666666666666667, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 5.166666666666667, 'instances_tested': 1, 'n_max': 5, 'conjecture_holds': True, 'counterexample': ''}

```

TRIAL: {'metric_name': 'CC_R(f)', 'metric_value': 6.333333333333333, 'instances_tested': 1, 'n_max': 40}
 RESULT: FALSIFIED counterexample="Function with 5 inputs and Function with 5 inputs and 23 leaves" fir

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers n up to 40, which is too small to conclude that the conjecture holds for all Boolean functions. The metric does not scale trivially with n, but a larger range of n values would be necessary to confirm or disprove the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test provides a counterexample where for a Boolean function with 5 inputs, the number of symplectic leaves exceeds the conjectured upper bound whi | next: To further investigate the conjecture, tests should be conducted with a wider range of n values to determine if the counterexample holds for larger functions or if it is an artifact of the specific function and input size tested.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13728
2	preregistration	ollama_remote	glm4:latest	0	0	9104
3	novelty	ollama_remote	glm4:latest	0	0	8604
4	novelty	ollama_remote	glm4:latest	0	0	12119
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12794
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33588
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9448
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9245
9	critic	ollama_remote	glm4:latest	0	0	38908
10	judge	ollama_remote	glm4:latest	0	0	19579

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167117 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e38b2feb290c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e38b2feb290c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e38b2feb290c.tar.gz (if generated)